Final Defense Dossier

Student: Gourik Narendra Patil

Date: 14 June 2026

Keep this compact. Use evidence bullets, not essays.

Day 1 - Debugging Discipline

Strongest debugging evidence:

- Verified network requests indicating frontend targeted port 8000 while backend ran on 5000, causing connection failures.
- Rejected hypothesis of "backend server down" by checking runtime health status and logs instead of assuming code errors.

What this proves: I rely on runtime evidence (network tabs, logs) before making code assumptions, correctly tracing configuration mismatches across boundaries.

Day 2 - Systems Reasoning

Strongest systems reasoning evidence:

- Diagnosed a status mismatch issue by identifying a Redis cache invalidation failure (18.5% cache mismatch rate).
- Prioritized disabling cache reads to restore source-of-truth consistency, despite the tradeoff of increased latency.

What this proves: I can separate downstream symptoms (UI bugs) from origin failures (cache poisoning) and prioritize data correctness over speed.

Day 3 - Readiness Verification

Readiness judgment evidence:

- Discovered a wrong `.env` file name breaking JWT authentication and a hardcoded constraint in `routes.js` blocking specific candidates.
- Verified the complete end-to-end recruiter workflow before declaring the system ready to ship.

What this proves: I do not declare a system "ready" based on a successful login alone; I challenge assumptions and test the full critical path.

Day 4 - Ambiguity Ownership

Ambiguity ownership evidence:

- Confronted undefined notification behavior for manual shortlisting by proposing a Minimal Safe V1.
- Delivered the requested recruiter control but explicitly blocked automated notifications to prevent communication mistakes.

What this proves: I can narrow vague requirements into safe iterations while rejecting undefined scope to protect the user experience.

Day 5 - AI Trust Judgment

AI trust judgment evidence:

- Reviewed an AI architecture plan and accepted the valid idea of storing `event_id` to prevent duplicate webhooks.
- Rejected the dangerous AI recommendation of using `payment_id` as the only idempotency key, noting it would ignore critical events like refunds.

What this proves: I do not blindly trust AI architectural advice without verifying state-transition safety and idempotency constraints.

Day 6 - Pressure Tradeoff

Pressure tradeoff evidence:

- Disabled payments during the LaunchRoom flash sale to contain broken checkout states and auto-refunded exact duplicates.
- Isolated the affected SKU and communicated transparently to users, prioritizing customer trust over immediate revenue recovery.

What this proves: Under time pressure, I prioritize containing systemic risk and protecting customer money over cosmetic fixes or leadership pressure.

Biggest Mistake This Week

What I got wrong: Initially assuming problems were caused by backend code (Day 1) or UI/notification systems (Day 2) without immediately checking runtime evidence.

What I learned: Upstream source-of-truth issues (cache, environment config) often mask themselves as downstream UI bugs.

What I would do differently now: Always check runtime network/logs and data consistency first before making assumptions about business logic.

Strongest Capability

My strongest capability: Systems Reasoning and Risk Containment

Evidence: Diagnosing the cache poisoning in Day 2 and isolating the affected SKU during the Day 6 simulation to protect the overall drop.

Weakest Capability

My weakest capability: Debugging speed at the onset of an incident.

Evidence: Spending initial time in Day 1 assuming backend issues instead of immediately checking the network tab.

Next 30-Day Engineering Growth Focus

Growth focus:

- Evidence-driven debugging
- Eliminating assumptions during early incident response

Daily/weekly behavior I will practice:

- Validating network requests before reading code
- Checking error logs as the absolute first step
- Verifying database/cache state boundaries before forming hypotheses